

Programação e Desenvolvimento de Software I

Estruturas

Prof. Héctor Azpúrua
(slides adaptados do Prof. Pedro Olmo)

Fast inverse square root

Quake



- Usado para calcular a incidência de luz nas superfícies e reflexões

```
float Q_rsqrt(float x) {  
    float xhalf = 0.5f * x;  
    int i = *(int *)&x;           // get bits for floating value  
    i = 0x5f375a86 - (i >> 1);    // gives initial guess y0  
    x = *(float *)&i;             // convert bits back to float  
    x = x * (1.5f - xhalf * x * x); // Newton step, repeating increases accuracy  
    return x;  
}
```

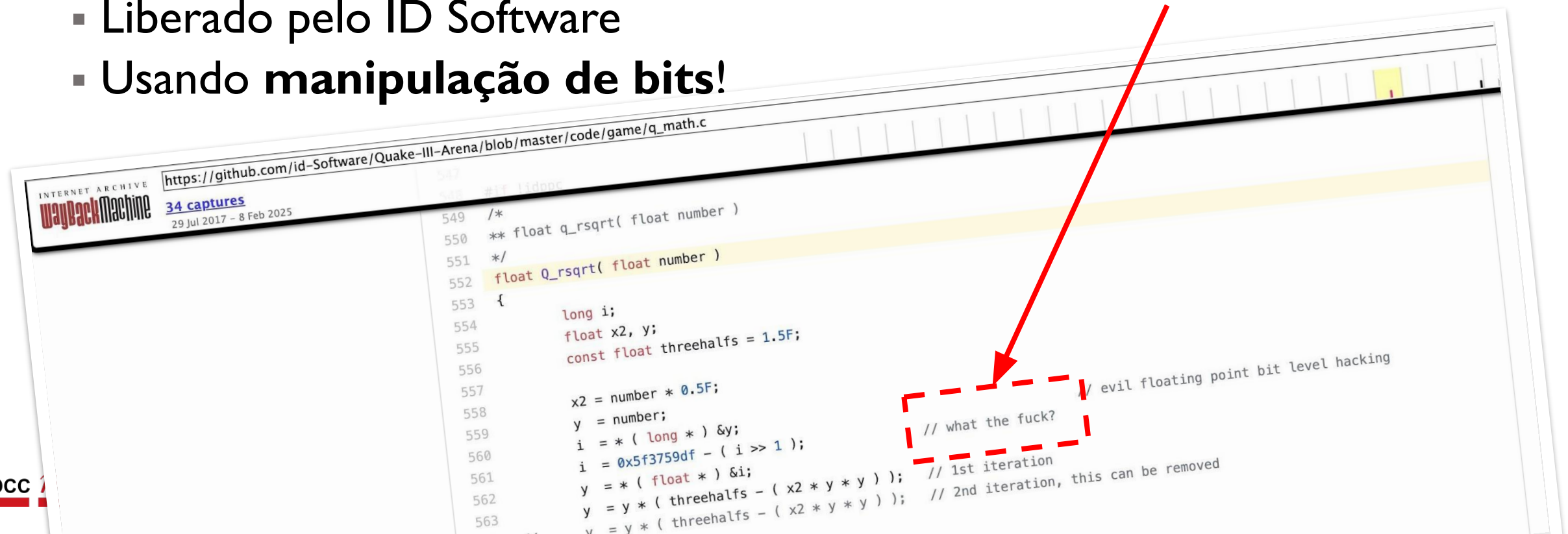
```
int main(int argc, char const *argv[]) {  
    float number = 9.1;  
    float res = Q_rsqrt(number);  
    printf("res1: %f\n", res);  
    printf("res2: %f\n", sqrt(1.0 / number));  
    return 0;  
}
```

```
./quake_fast_sqrt  
res1: 0.331091  
res2: 0.331497
```

Fast inverse square root

Quake

- Diretamente do código do Quake III
 - https://github.com/id-Software/Quake-III-Arena/blob/master/code/game/q_math.c#L552
 - Liberado pelo ID Software
 - Usando **manipulação de bits!**



Estruturas

Problema I

- Implementar uma calculadora para fazer operações sobre frações (ex: $1/3$, $5/13$, etc.)
- A calculadora deve ser capaz de realizar as seguintes operações:
 - Somar
 - Dividir
 - Subtrair
 - Multiplicar

Estruturas

Problema 1

```
void somar_fracoas(int a, int b, int c, int d);  
void subtrair_fracoas(int a, int b, int c, int d);  
void multiplicar_fracoas(int a, int b, int c, int d);  
void dividir_fracoas(int a, int b, int c, int d);
```

```
int main(int argc, char const *argv[]) {  
    char op;  
    int a, b, c, d;  
  
    while (1) {  
        printf("1. Somar\n");  
        printf("2. Subtrair\n");  
        printf("3. Multiplicar\n");  
        printf("4. Dividir\n");  
        printf("Escolha uma opcao, ou 0 para sair:");  
        scanf("%c", &op);  
  
        op = toupper(op);  
        if (op == '0')  
            break;  
  
        printf("Digite os valores de 'a/b': ");  
        scanf("%d/%d", &a, &b);  
        printf("\nDigite os valores de 'c/d': ");  
        scanf("%d/%d", &c, &d);  
  
        switch (op) {  
            case '1':  
                somar_fracoas(a, b, c, d);  
                break;  
            case '2':  
                subtrair_fracoas(a, b, c, d);  
                break;  
            case '3':  
                multiplicar_fracoas(a, b, c, d);  
                break;  
            case '4':  
                dividir_fracoas(a, b, c, d);  
                break;  
            default:  
                printf("Opcao invalida\n");  
        }  
    }  
    return 0;  
}
```

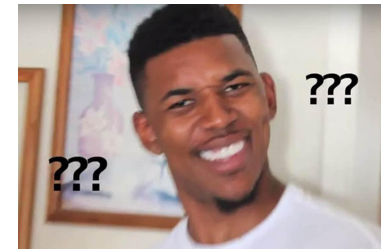
Estruturas

Problema I

- Qual é o problema deste código?

```
void somar_fracoes(int a, int b, int c, int d);  
void subtrair_fracoes(int a, int b, int c, int d);  
void multiplicar_fracoes(int a, int b, int c, int d);  
void dividir_fracoes(int a, int b, int c, int d);
```

- É confuso...
 - Possui muitos parâmetros e não fica claro quem é quem nos parâmetros
- Podemos fazer melhor!
 - E se uma fração poderia ser representada com seu **próprio tipo de dados**?
 - Um novo tipo criado e controlado por nós



Estruturas

Problema I

É definido um novo tipo de dados!

O tipo `frac` (usando `struct` e `typedef`)

O tipo `frac` pode ser usado como parâmetro ou tipo de retorno em funções

```
typedef struct {  
    int num;  
    int den;  
} frac;
```

```
void obter_fracoes(frac* f1, frac* f2);  
void somar_fracoes(frac* f1, frac* f2);  
void subtrair_fracoes(frac* f1, frac* f2);  
void multiplicar_fracoes(frac* f1, frac* f2);  
void dividir_fracoes(frac* f1, frac* f2);
```

```
frac a, b;  
obter_fracoes(&a, &b);  
  
...  
  
switch (op) {  
    case '1':  
        somar_fracoes(&a, &b);  
        break;  
    case '2':  
        subtrair_fracoes(&a, &b);  
        break;  
    case '3':  
        multiplicar_fracoes(&a, &b);  
        break;  
    case '4':  
        dividir_fracoes(&a, &b);  
        break;  
    default:  
        printf("Opcao invalida!\n");  
}
```

Definição de novos tipos de dados

Structs

- Se cada fração compreende dois inteiros, como é possível fazer uma função para somar duas frações passando apenas dois parâmetros?
- Isto é possível porque a linguagem C permite a definição de **novos tipos de dados** com base **nos tipos primitivos**:
 - `char`, `int`, `float` e `double`
- Estes novos tipos de dados, formados a partir dos tipos primitivos são chamados de **tipos estruturados**
 - Ou **estruturas/structs**!

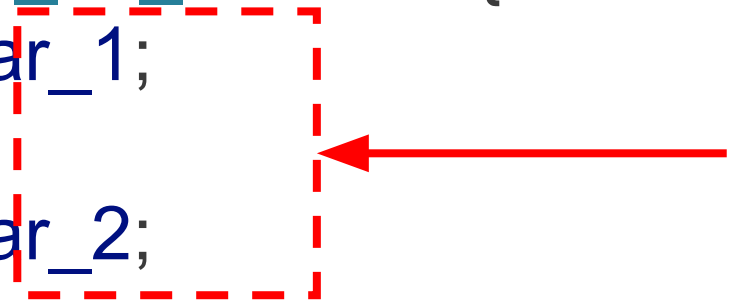
Definição de novos tipos de dados

Structs

- Uma variável de um determinado tipo estruturado definido pelo usuário é comumente chamada de uma estrutura
- Uma estrutura **agrupa várias variáveis** de diversos tipos em uma só variável
- Para criar uma estrutura usa-se o comando `struct`:

```
struct nome_da_estrutura{  
    tipo_1 var_1;  
    ...  
    tipo_2 var_2;  
};
```

As variáveis que compõem a estrutura são chamadas de **campos da estrutura**



Exemplos de estruturas

- A declaração de variáveis de um tipo estruturado (estruturas) é feita da mesma forma que para um tipo simples
- Estruturas podem ter como campos **outras estruturas!**

```
struct ponto {  
    float coord_x;  
    float coord_y;  
};  
  
struct circulo {  
    float raio;  
    struct ponto centro;  
};  
  
struct cilindro {  
    float altura;  
    struct circulo base;  
};
```

- A inicialização de uma estrutura é feita da mesma forma que uma variável normal:

```
struct ponto p1;  
p1.coord_x = 1.0;  
p1.coord_y = 2.0;
```

Exemplos de estruturas

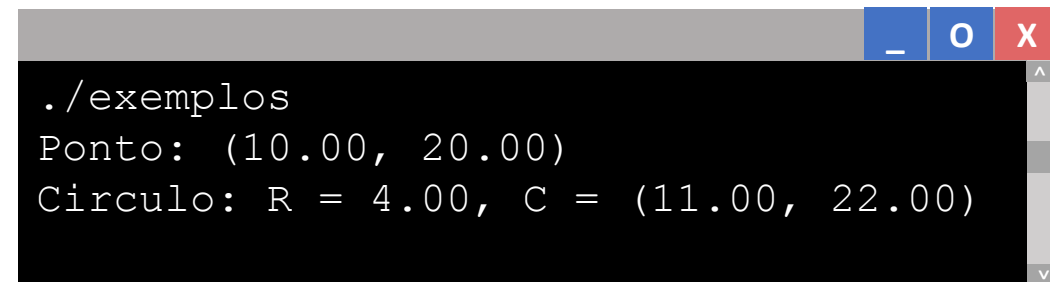
Inicialização com { }

```
struct circulo {  
    float raio;  
    struct ponto centro;  
};
```

- Podemos definir estruturas com valores iniciais numa mesma operação utilizando { }
- De forma similar a como definimos um array **pre-preenchido** com valores
- Cuidado!** A ordem de preenchimento vem dada pela ordem dos parâmetros na definição da estrutura

```
struct ponto p1 = {10.0, 20.0};  
printf("Ponto: (%.2f, %.2f)\n",  
       p1.coord_x,  
       p1.coord_y);
```

```
struct circulo c1 = {4.0, 11.0, 22.0};  
printf("Circulo: R = %.2f, C = (%.2f, %.2f)\n",  
       c1.raio,  
       c1.centro.coord_x,  
       c1.centro.coord_y);
```



```
./exemplos  
Ponto: (10.00, 20.00)  
Circulo: R = 4.00, C = (11.00, 22.00)
```

Exemplos de estruturas

Acesso aos campos

- Para se acessar os campos de uma estrutura, basta separar o nome da variável pelo símbolo ponto (.)

```
struct ponto p;  
p.coord_x = 1.0;  
p.coord_y = 2.0;
```

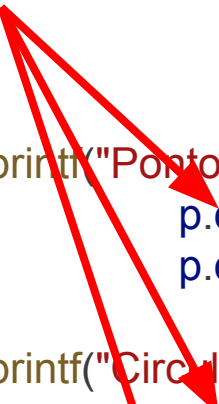
```
struct circulo c;  
c.raio = 4.0;  
c.centro = p;
```

```
struct cilindro d;  
d.altura = 3.0;  
d.base = c;
```

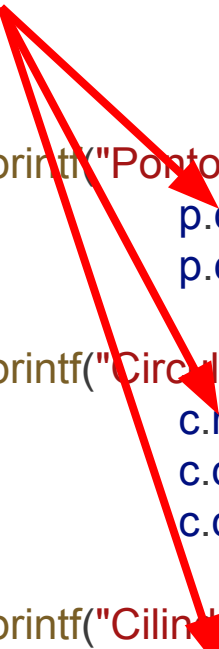
```
printf("Ponto: (%.2f, %.2f)\n",  
      p.coord_x,  
      p.coord_y);
```



```
printf("Circulo: Raio = %.2f, Centro = (%.2f, %.2f)\n",  
      c.raio,  
      c.centro.coord_x,  
      c.centro.coord_y);
```



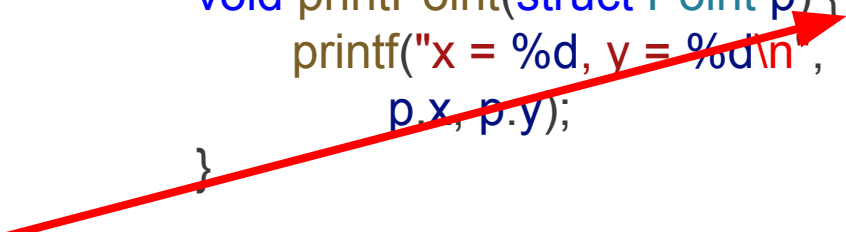
```
printf("Cilindro: Altura = %.2f, Base Raio = %.2f\n",  
      d.altura,  
      d.base.raio);
```



Passando estruturas como parâmetro

- Podemos passar estruturas como parâmetros:
 - Via valor (copia)
 - Usando uma referência (ponteiro)
- Assinatura da função deve ser:
`struct nome_da_estrutura var;`
- Como podemos fazer melhor?
 - **Diminuir a complexidade na definição**

```
struct Point {  
    int x;  
    int y;  
};  
  
void printPoint(struct Point p) {  
    printf("x = %d, y = %d\n",  
        p.x, p.y);  
}  
  
int main() {  
    struct Point pt = {10, 20};  
    printPoint(pt);  
    return 0;  
}
```



O comando `typedef`

Redefinir um tipo

- O Comando `typedef` permite ao programador definir um novo nome para um determinado tipo
- Sua forma geral é:

`typedef` nome_antigo nome_novo;

- Exemplo:
 - Dando o nome inteiro para o tipo int:

```
typedef int inteiro_cachorro;  
inteiro_cachorro num;
```

Isso aqui é bom? 🤔

Tem que usar o **bom senso!**

O comando typedef

Opinião do Linus Torvalds (criador o kernel do Linux)

- <https://yarchive.net/comp/linux/typedefs.html>
- ***The linux coding style tends to avoid using typedefs. It's not a hard rule '...' but it's fairly common to use an explicit "struct xxxx" instead of "xxxx_t".***

Newsgroups: fa.linux.kernel
From: Linus Torvalds <torvalds@transmeta.com>
Subject: Re: [PATCH] 2.5.21 - list.h cleanup
Original-Message-ID: <Pine.LNX.4.44.0206100954250.30535-100000@home.transmeta.com>
Date: Mon, 10 Jun 2002 17:03:50 GMT
Message-ID: <fa.l4d7opv.1mhk21p@ifi.uio.no>

On Mon, 10 Jun 2002, Thomas 'Dent' Mirlacher wrote:

>
> ok, the point that *_t is hiding implementation details (when used for
> structs is valid). but is there a general consens on not using typedefs
> for structs?

~~The linux coding style tends to avoid using typedefs. It's not a hard
rule (and I did in fact apply this patch, since it otherwise looked fine),
but it's fairly common to use an explicit "struct xxxx" instead of
"xxxx_t".~~

I dislike type abstraction if it has no real reason. And saving on typing
is not a good reason - if your typing speed is the main issue when you're
coding, you're doing something seriously wrong.

(Similarly, if you are trying to compress lines to be shorter, you have
nothing to do with type names).

O comando `typedef`

Dando nomes a tipos complexos

- O comando `typedef` também pode ser utilizado para dar nome a tipos complexos como estruturas
- Exemplos:

```
typedef struct tipo_endereco {  
    char rua[50];  
    int numero;  
    char bairro[20];  
    char cidade[30];  
    char sigla_estado[3];  
    long int CEP;  
} TEndereco;
```

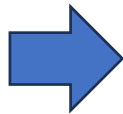
```
typedef struct frac {  
    int num;  
    int den;  
} frac;
```


O comando `typedef`

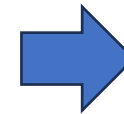
Dando nomes a tipos complexos

- Utilizando-se o comando `struct` juntamente com o comando `typedef`, pode-se dispensar o uso da palavra `struct` na declaração da variável

```
struct ponto {  
    int x;  
    int y;  
};
```



```
typedef struct ponto {  
    float x;  
    float y;  
} ponto;
```



```
typedef struct {  
    float x;  
    float y;  
} ponto;
```

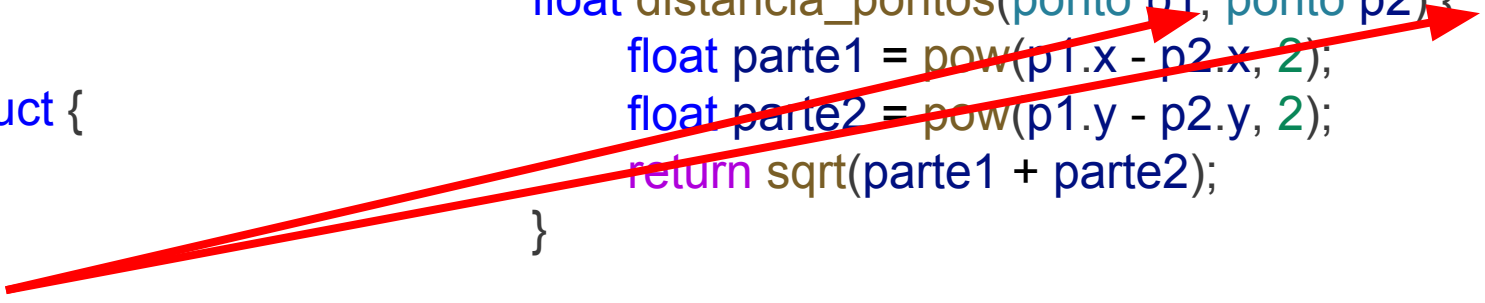


O comando `typedef`

Estruturas como parâmetros de funções

- Com um novo tipo podemos simplificar a forma de passar as estruturas como parâmetros

```
typedef struct {  
    float x;  
    float y;  
} ponto;  
  
float distancia_pontos(ponto p1, ponto p2) {  
    float parte1 = pow(p1.x - p2.x, 2);  
    float parte2 = pow(p1.y - p2.y, 2);  
    return sqrt(parte1 + parte2);  
}  
  
int main() {  
    ponto u, v;  
    scanf("%f %f %f %f", &u.x, &u.y, &v.x, &v.y);  
    printf("\n %f\n", distancia_pontos(u, v));  
}
```



Passando estruturas por referência

- E como faço para passar estruturas por referência?
 - Igual que quando passamos por referência usando variáveis normais
 - Lembrar dos operadores: **&** e *****

```
typedef struct ponto {  
    float x;  
    float y;  
} ponto;  
  
void le_coordenada(ponto *p1) {  
    float a, b;  
    printf("Digite a coordenada x e y\n");  
    scanf("%f %f", &a, &b);  
    p1->x = a;  
    *(p1).y = b;
```

```
void main() {  
    ponto u, v;  
  
    le_coordenada(&u);  
    le_coordenada(&v);  
  
    printf("\n %f\n",  
           distancia_pontos(u, v));  
}
```

Passando estruturas por referência

Acesso aos campos

- Para se acessar os campos de uma estrutura passada por referência:
 - Diretamente com o operador `->` (setinha)
 - De referenciar com `*` e acessar os campos usando o símbolo ponto `(.)`

```
void printa_ponto(ponto* p) {  
    printf("Ponto: (%.2f, %.2f)\n", p->x, p->y);  
}
```

```
struct ponto p;  
p.coord_x = 1.0;  
p.coord_y = 2.0;
```

```
void printa_ponto2(ponto* p) {  
    printf("Ponto: (%.2f, %.2f)\n", (*p).x, (*p).y);  
}
```

```
printa_ponto(&p);  
printa_ponto(&p);
```

Passando estruturas por referência

Exemplo passo a passo

```
void main() {  
    ponto u, v;  
    le_coordenada(&u);  
    le_coordenada(&v);  
    printf("\n %f\n", distancia_pontos(u, v));  
    getchar();  
}
```

Endereço	Variável	Conteúdo

Passando estruturas por referência

Exemplo passo a passo

```
void main() {  
    ponto u, v;  
    le_coordenada(&u);  
    le_coordenada(&v);  
    printf("\n %f\n", distancia_pontos(u, v));  
    getchar();  
}
```

Endereço	Variável	Conteúdo
0xF000	u.x	
0xF010	u.y	
0xF020	v.x	
0xF030	v.y	
0xF040		
0xF050		
0xF060		

Passando estruturas por referência

Exemplo passo a passo

```
void main() {  
    ponto u, v;  
    le_coordenada(&u);  
    le_coordenada(&v);  
    printf("\n %f\n", distancia_pontos(u, v));  
    getchar();  
}
```

← Chamar a função

Endereço	Variável	Conteúdo
0xF000	u.x	
0xF010	u.y	
0xF020	v.x	
0xF030	v.y	
0xF040		
0xF050		
0xF060		

Passando estruturas por referência

Exemplo passo a passo

```
le_coordenada(&u);    -- ➡ void le_coordenada(ponto *p1) {  
                        float a, b;  
                        printf("Digite a coordenada x e y\n");  
                        scanf("%f %f", &a, &b);  
                        p1->x = a;  
                        p1->y = b;  
                        }
```

Endereço	Variável	Conteúdo
0xF000	u.x	
0xF010	u.y	
0xF020	v.x	
0xF030	v.y	
0xF040		
0xF050		
0xF060		

Passando estruturas por referência

Exemplo passo a passo

le_coordenada(&u);

-- ➡ void le_coordenada(ponto *p1) {
float a, b;
printf("Digite a coordenada x e y\n");
scanf("%f %f", &a, &b);
p1->x = a;
p1->y = b;
}

Endereço	Variável	Conteúdo
0xF000	u.x	
0xF010	u.y	
0xF020	v.x	
0xF030	v.y	
0xF040	p1	0xF000
0xF050	a	1.5
0xF060	b	20

Similar a vetores, &u dá o endereço inicial que a estrutura está armazenada

Passando estruturas por referência

Exemplo passo a passo

```
le_coordenada(&u);    -- ➡ void le_coordenada(ponto *p1) {  
                        float a, b;  
                        printf("Digite a coordenada x e y\n");  
                        scanf("%f %f", &a, &b);  
                        p1->x = a;  
                        p1->y = b;  
                    }
```

Endereço	Variável	Conteúdo
0xF000	u.x	1.5
0xF010	u.y	2.0
0xF020	v.x	
0xF030	v.y	
0xF040	p1	0xF000
0xF050	a	1.5
0xF060	b	2.0

Passando estruturas por referência

Exemplo passo a passo

```
void main() {  
    ponto u, v;  
    le_coordenada(&u);  
    le_coordenada(&v);  
    printf("\n %f\n", distancia_pontos(u, v));  
    getchar();  
}
```

← Chamar a função

Endereço	Variável	Conteúdo
0xF000	u.x	1.5
0xF010	u.y	2.0
0xF020	v.x	
0xF030	v.y	
0xF040		
0xF050		
0xF060		

Passando estruturas por referência

Exemplo passo a passo

```
le_coordenada(&v);    -- -- ➡ void le_coordenada(ponto *p1) {  
                        float a, b;  
                        printf("Digite a coordenada x e y\n");  
                        scanf("%f %f", &a, &b);  
                        p1->x = a;  
                        p1->y = b;  
                    }
```

Endereço	Variável	Conteúdo
0xF000	u.x	1.5
0xF010	u.y	2.0
0xF020	v.x	
0xF030	v.y	
0xF040	p1	0xF020
0xF050	a	30
0xF060	b	25.5

Passando estruturas por referência

Exemplo passo a passo

```
le_coordenada(&v);    -- ➡ void le_coordenada(ponto *p1) {  
                        float a, b;  
                        printf("Digite a coordenada x e y\n");  
                        scanf("%f %f", &a, &b);  
                        p1->x = a;  
                        p1->y = b;  
                    }
```

Endereço	Variável	Conteúdo
0xF000	u.x	1.5
0xF010	u.y	2.0
0xF020	v.x	30
0xF030	v.y	25.5
0xF040	p1	0xF020
0xF050	a	30
0xF060	b	25.5

Passando estruturas por referência

Exemplo passo a passo

```
void main() {  
    ponto u, v;  
    le_coordenada(&u);  
    le_coordenada(&v);  
    printf("\n %f\n", distancia_pontos(u, v));  
    getchar();  
}
```

Endereço	Variável	Conteúdo
0xF000	u.x	1.5
0xF010	u.y	2.0
0xF020	v.x	30.0
0xF030	v.y	25.5
0xF040		
0xF050		
0xF060		

Perguntas?

- E-mail:
 - hector@dcc.ufmg.br
- Material da disciplina:
 - <https://pedroolmo.github.io/teaching/pdsI.html>
- Github:
 - <https://github.com/h3ct0r>



Héctor Azpúrua
h3ct0r